



4WD Torque Vectoring Platform

Implementation Tracker & Action Plan

A phased, tick-as-you-go checklist for building the modular Simulink control development platform — from an empty repository to a 4WD-capable, version-controlled tool.

UNIVERSITY OF STRATHCLYDE MOTORSPORT

Companion to: *4WD Torque Vectoring — A Modular Control Development Platform*

N. Kostomoudis · 20 June 2026 · v1.0

How to use this tracker

Current status — 23 June 2026 (v1.2)

Phases 0–2 complete (tags `v0.1-skeleton`, `v0.2-skeleton-runs`, `v0.3-baseline-loop`). The thesis is reproduced inside the clean modular platform: linear bicycle plant, understeer reference, PI controller, baseline allocator (config ladder), and a manoeuvre generator feeding one coherent closed loop — now with a working metric harness (RMSE / IAE / peak). **This is the first real summit. Next: Phase 3** (steps 14–17) — the double-track plant, per-wheel tyres + load transfer, the constraint-aware allocator, and the manoeuvre/metric expansion that make it a genuine 4WD tool.

This is a working checklist, not a spec — the spec is the companion document. Work the items in order; each is sized to be a single focused sitting, and the job is ‘done properly’ before moving on. Tick the box as you complete each one (print it, or check it off in your PDF reader / regenerate with the box ticked).

The plan is organised so that:

- **The system is runnable after every phase** — never a long stretch with nothing that works. Phase 1 gives a loop that runs end-to-end before any real physics exists.
- **Milestones map to git tags**. At each milestone you tag a release and (at major ones) regenerate the spec PDF, so every documented version corresponds to a recoverable code state.
- **Phases 0–2 alone are a complete deliverable** — the thesis reproduced in a clean, modular, version-controlled structure. That is the first real summit (tag `v0.3`); everything after is genuine extension.

Scope discipline for a free-time project

The risk is not doing it badly — it is stalling mid-way through the harder physics (Phase 3). Treat `v0.3` as the first complete, shareable result and a clean handover point. Reaching it properly is worth more than rushing toward 4WD and leaving a half-built model nobody else can run.

Five guiding principles

■ Simulink-first, code where it helps

The control system lives in Simulink (visual, modular, learnable). Parameters, bus definitions, metrics, and config live in tracked `.m` files — the parts version control can actually diff and merge.

■ Bus objects are the interface contracts

Every signal crossing a block boundary is a defined `simulink.Bus`. This is how the modularity from the spec becomes real: a block can be swapped as long as it honours its bus. Get this right early.

■ Referenced models, not one monolith

Each major block is its own referenced `.slx`. People edit different files, sidestepping the binary-merge problem, and blocks become independently swappable.

■ Walking skeleton before real physics

4WD TV Platform — Implementation Tracker

Wire the whole loop with passthrough placeholder blocks first; then fill each block one at a time, always keeping a runnable system. Far less error-prone than integrating finished blocks at the end.

■ Port the proven thesis pieces

Build the architecture from scratch, but harvest the thesis's reference logic, PI/fuzzy controllers, and metric calculations into the new block structure once the skeleton exists.

PHASE 0

Goal: a clean, version-controlled, empty project

Foundations — repository & version control

Before any model exists, get the plumbing right. This is the cheapest, highest-leverage phase: doing it now avoids re-doing everything under version control later.

**1** Install Git and connect it to MATLAB.

Install Git for your OS. In MATLAB, confirm Source Control integration works (Current Folder → right-click → Source Control). On Windows, note Git hooks may need Cygwin on the MATLAB library path for commit/merge.

**2** Create the GitHub repo and a MATLAB Project inside it.

Create a private GitHub repo. Clone locally over SSH (set up an SSH key to avoid login prompts). Create a MATLAB Project (.prj) in the cloned folder — the Project is what makes Simulink + Git cooperate (dependency tracking, derived-file warnings).

**3** Add .gitignore and .gitattributes, and commit them FIRST.

Use the versions from the spec (§5): ignore slprj/, *.slxc, *.asv, work/, codegen/; mark .slx .mlx .mat .fig as binary. Committing these before any model files exist is the classic thing people get wrong — do it now.

**4** Write skeleton README.md and start CHANGELOG.md; first tagged commit.

README: one paragraph on what the project is and how to open it. CHANGELOG: start the running log (date + what changed). Commit, then tag v0.1-skeleton.

v0.1-skeleton**• MILESTONE — empty but version-controlled project, ready for models — REACHED**

PHASE 1

Goal: the whole loop runs end-to-end (with empty blocks)

Interface contracts & the walking skeleton

Define the contracts that make the platform modular, then wire a complete — if hollow — signal loop. After this phase you have a runnable system; everything later just fills in blocks.

**5 Define the Simulink.Bus objects for every interface. (Keystone task.)**

In a tracked `defineBuses.m`, define the buses for each boundary from spec §4.1: reference→controller, controller→allocator, allocator→powertrain, powertrain→plant, plant→sensors. List the exact signals in each. This is the single most important early decision — everything downstream depends on it.

**6 Write `vehicleConfig.m` returning the config struct.**

Geometry, mass, tyre params, and the actuator field for the ladder (RWD_single / RWD_twin / AWD_4), plus an arch field (controller, allocator, topology). Tag the parameters that are still estimates (h , l_z , μ).

**7 Build the walking skeleton: top-level model with passthrough blocks.**

A top-level `.slx` with each block as an empty/passthrough referenced subsystem, wired with the buses, runnable end-to-end and producing a trivial output (e.g. a constant or a fed-through signal on a scope). Confirm it simulates with no errors.

v0.2-skeleton-runs**• MILESTONE — end-to-end loop simulates; interfaces fixed — REACHED****Why the keystone matters**

If the buses are right, every later block swaps cleanly and people can work in parallel on different referenced models. If they are wrong, you discover it during integration and rewire everything. Spend real time on step 5 — sketch the signal list on paper first.

PHASE 2

**Fill the blocks — reproduce the thesis,
modularly**Goal: the thesis baseline, rebuilt in the new
structure

Fill each placeholder with real (initially simple) content, one per sitting, always keeping the loop runnable. By the end you have reproduced the thesis result inside the clean modular architecture — the first real summit.

**8** Plant block v1: port the linear single-track (bicycle) model.

Ported as `bicycleModel.m` in a MATLAB Function block with two external integrators (v_y , r). Step-steer response verified (~ 0.31 rad/s to 0.05 rad at $v_x=10$).

**9** Reference generator: port the understeer-based reference yaw logic.

Built as `referenceGen.m`: $r_{ref} = v \cdot \delta / (L + K_u \cdot v^2)$ with friction-limit clamp. Steady-state target 0.2972 rad/s, matches hand-calc.

**10** Controller block: port the thesis PI controller.

Built with the native PID Controller block (PI mode), gains K_p/K_i from `cfg`, saturation + anti-windup at Mz_{Max} . Unpacks `RefBus` + `PlantBus` to form the error — first block to use its bus inputs.

**11** Allocator block: moment-to-torque split with config-ladder topology.

Built as `allocateBaseline.m` (chose the real rule-based split, not pass-through). $dT = Mz \cdot R_w / tr$; `apply_topology` realises the ladder (`RWD_single` / `RWD_twin` / `AWD_4`). Verified numerically for all three rungs. NB: torque not yet consumed by the linear plant — coupling arrives with the Phase 3 double-track plant.

**12** Manoeuvre generator + steering routing (`DriverBus`).

Added a 6th interface contract `DriverBus` (δ) and a Manoeuvre referenced model (step steer). Pulled the internal step-steer out of `Plant` and `Reference`; one steering source now feeds both via `DriverIn` inputs. Full closed loop is coherent and runs clean.

**13** Metric harness: IAE / RMSE / peak into `metrics.m`.

Built `metrics.m`: logs reference + actual yaw rate, computes RMSE / IAE / peak error + steady-state checks. Verified: r_{Ref} 0.297 , r_{Act} 0.313 , RMSE 0.018 , peak error 0.297 (the step transient). The platform can now *measure* tracking.

v0.3-baseline-loop

**• MILESTONE — thesis reproduced, modular + version-controlled — REACHED
(first real summit)**

Regenerate the spec PDF at this milestone and tag the release. This is the first complete, shareable deliverable and a clean handover point.

PHASE 3

Make it a 4WD tool

Goal: per-wheel plant + real allocation + the
config ladder

The structural fidelity jump that turns a yaw-tracking demo into a 4WD development platform. This is the hardest phase — pace it, and keep the loop runnable after each step.

☐ **14** Upgrade the plant to double-track (four corners).

Replace the single-track plant with a four-corner model carrying independent per-wheel vertical loads. Same plant bus, higher fidelity inside — nothing else changes.

☐ **15** Add per-wheel nonlinear (saturating) tyre + lateral load transfer.

A Fiala / simplified Magic-Formula tyre per corner (clips at $\mu \cdot F_z$), and $\Delta F_z = m \cdot a_y \cdot h / tr$ feeding the corner loads. Watch for solver chatter at the saturation knee — smooth the tyre curve if it appears.

☐ **16** Build the real allocator (pseudo-inverse) + `apply_topology`.

Demand $[M_z; F_x] \rightarrow$ four wheel torques via a weighted pseudo-inverse. `apply_topology` zeroes undriven wheels so `RWD_single` / `RWD_twin` / `AWD_4` all run the same code — the config ladder in action.

☐ **17** Add remaining manoeuvres + expand the metric set.

Skidpad, slalom, ramp steer. Add the 4WD metrics: per-wheel tyre utilisation, allocation distribution, constraint margin, and energy per manoeuvre.

v0.4-4wd-capable

• MILESTONE — double-track plant, real allocation, 4WD metrics

Regenerate the spec PDF — the platform is now genuinely 4WD-capable.

PHASE 4

Breadth & validationGoal: comparison grid, powertrain realism,
real-data anchor

As time allows — these turn the tool from capable to trustworthy and broad.

- ☐ **18** Port the fuzzy controller; set up the architecture comparison grid.

Second controller behind the same bus. Loop over {controller} × {allocator} on a fixed manoeuvre/config and tabulate the metric set — the evidence for architecture selection.
- ☐ **19** Add the powertrain block (envelope, lag, limits).

Between allocator and plant, honouring the powertrain bus: torque-speed envelope, actuator lag, power and regen caps. No other block changes.
- ☐ **20** Telemetry-replay mode + twin-motor correlation.

Feed measured steering/speed through the RWD_twin configuration; overlay simulated vs. measured yaw rate against the real data. The credibility anchor for 4WD extrapolation.
- ☐ **21** Longitudinal-control module (the modular-growth test).

Traction/ABS sharing the allocator and plant via existing buses. If it slots in without touching the yaw controller, the architecture was right.

v0.5

• MILESTONE — comparison grid + powertrain + correlation + longitudinal

PHASE 5

Python & external integrationGoal: bring in solvers / big-data tools only when
needed

- ☐ **22** Integrate Python only when a real need appears.

When an optimisation-based allocator, an external solver, or large-scale data handling demands it, integrate Python via MATLAB pyenv/pyrun — kept behind the same bus contract so it is a drop-in block, not a rewrite. Do not pull this forward before the need is concrete.

Resist premature integration

Python adds an environment dependency that complicates setup for the next person. Keep the platform pure-MATLAB until a task genuinely needs more — then add Python for that one block only.

Documentation cadence

Two complementary layers keep the process documented without making documentation a chore. The lightweight layer runs continuously; the heavy layer is a milestone snapshot.

Layer	When	What
CHANGELOG.md	Every meaningful commit	One line: date + what changed + why. Cheap, continuous, lives in the repo. The running narrative of the project.
Git tag / release	Each milestone	Tag the commit (v0.1 ... v0.5) so each documented state is recoverable. Add a short release note summarising the milestone.
Spec PDF regenerate	Major milestones only	Regenerate the companion spec at v0.3, v0.4, v0.5. The polished snapshot — do NOT do this every change, or it gets abandoned.
Tracker (this doc)	As you go	Tick items off. Optionally regenerate with boxes checked at each milestone for a visual progress record.

The summit map

If you remember nothing else: the milestones are the real structure. Each is a complete, recoverable, shareable state — aim for the next one, not the final output.

v0.1	Version-controlled empty project
v0.2	Loop runs end-to-end (hollow)
v0.3	Thesis reproduced, modular — FIRST REAL SUMMIT
v0.4	4WD-capable (per-wheel plant + allocation)
v0.5	Broad + validated (grid, powertrain, correlation)

Next: we take item 1 — installing Git and wiring up MATLAB's source control — and work through it step by step, missing nothing. Then item by item from there.